

# Proyecto Final

Aplicación para Resolver un Problema Real

---

## DESCRIPCIÓN E INSTRUCCIONES

Primer Semestre 2026

|                           |                                         |
|---------------------------|-----------------------------------------|
| <b>Curso:</b>             | Fundamentos de Programación – Sección C |
| <b>Catedrático:</b>       | Enrique Andrés Bolaños Reyes            |
| <b>Auxiliar:</b>          | Daniel Ernesto Hernández Zelada         |
| <b>Publicación:</b>       | Jueves 23 de abril de 2026              |
| <b>Entrega y defensa:</b> | Jueves 14 de mayo de 2026               |
| <b>Lugar:</b>             | Auditorio Hayek                         |
| <b>Modalidad:</b>         | Grupos de 2–3 personas                  |
| <b>Plataforma:</b>        | MIU (código + link de demo si aplica)   |
| <b>Punteo:</b>            | 30 puntos                               |

UNIVERSIDAD FRANCISCO MARROQUÍN  
Facultad de Ciencias Económicas

## 1. Descripción

El Proyecto Final es el **entregable integrador** del curso: una aplicación en **Python** que resuelve un *problema real* —tuyo, de un negocio, de una ONG, de tu familia, de tu comunidad— usando **todo** lo aprendido durante el semestre.

A diferencia del Proyecto 1 (reglas fijas impuestas por mí) y del Proyecto 2 (herramienta *no-code* con *scope* cerrado), aquí vos **elegís el problema, el diseño y el alcance**. Yo no te voy a decir qué construir; te voy a evaluar si lo que construiste **resuelve algo** y si demostrás los conceptos mínimos del curso.

### Tu proyecto debe:

- Resolver un problema **concreto y verificable** (no “hago una app de todo”).
- Funcionar **end-to-end** —desde que se ejecuta hasta que produce un resultado útil.
- Ser **demostrable en vivo** en 5–7 minutos en el Auditorio Hayek.
- Estar en **GitHub**, con *commits* reales tuyos a lo largo de la semana.

## 2. Ideas de Proyecto

No estás obligado a elegir de esta lista — sirve como **punto de partida**. Lo ideal es que identifiques un problema *tuyo* o de *alguien cercano* y lo resuelvas.

| Categoría                  | Ejemplo concreto                                                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Análisis de datos</b>   | Dashboard en consola que lee un CSV de gastos personales/ventas y reporta tendencias, top categorías, anomalías.             |
| <b>Consumo de API</b>      | App que consulta una API pública (clima, criptos, Spotify, PokéAPI, Banguat tipo de cambio) y genera recomendaciones.        |
| <b>Gestión / CRUD</b>      | Sistema de inventario, agenda de citas, biblioteca personal, control de tareas con persistencia en archivo.                  |
| <b>Juego</b>               | Trivia con categorías leídas desde JSON, juego por turnos con puntajes persistentes, aventura de texto con estado guardable. |
| <b>Automatización</b>      | Script que renombra/ordena archivos, genera reportes PDF/Excel a partir de datos, scrape ligero de una página pública.       |
| <b>NLP / AI básico</b>     | Analizador de sentimiento simple, clasificador de correos spam, chatbot de reglas con respuestas a partir de un archivo.     |
| <b>Finanzas personales</b> | Calculadora de inversión con metas, simulador de préstamos, seguimiento de presupuesto mensual.                              |
| <b>Educación</b>           | Flashcards con algoritmo de repetición espaciada, quiz con preguntas guardadas, generador de ejercicios.                     |

**Regla:** si no podés explicar en una frase “*mi programa sirve para \_\_\_\_\_*”, el problema todavía no está claro.

### 3. Requisitos Técnicos Mínimos

El código **debe demostrar**, en alguna parte del proyecto, los siguientes elementos. Esto no es opcional: la rúbrica evalúa su presencia.

| # | Elemento                                    | Qué debe demostrar                                                                                                                    |
|---|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Programación Orientada a Objetos</b>     | Al menos <b>1 clase propia</b> con atributos y métodos. Instanciación y uso real (no una clase decorativa vacía).                     |
| 2 | <b>Funciones</b>                            | Mínimo <b>5 funciones</b> propias con parámetros y <b>return</b> . Código modular, no un solo bloque gigante.                         |
| 3 | <b>Estructuras de control</b>               | <b>if / elif / else, for, while</b> usados con propósito.                                                                             |
| 4 | <b>Estructuras de datos</b>                 | Al menos <b>2 de</b> : listas, diccionarios, tuplas, sets. Elegir la adecuada al problema.                                            |
| 5 | <b>Manejo de archivos o consumo de APIs</b> | Leer/escribir <b>.csv, .json, .txt</b> o consumir una API con <b>requests</b> . Preferiblemente ambos.                                |
| 6 | <b>Manejo de errores</b>                    | <b>try / except</b> para entradas inválidas, archivos faltantes, fallos de red. No debe reventar en casos comunes.                    |
| 7 | <b>Recursividad</b> <i>donde aplique</i>    | Una función recursiva donde tenga sentido (árbol, búsqueda, <i>factorial</i> , Fibonacci con memoización, navegación en estructuras). |
| 8 | <b>Git + GitHub</b>                         | Repositorio público con <b>varios commits reales</b> a lo largo del desarrollo (no un solo <i>“initial commit”</i> ).                 |
| 9 | <b>Documentación</b>                        | <b>README.md</b> con: qué hace, cómo instalarlo, cómo ejecutarlo, ejemplo de uso. <i>Docstrings</i> en funciones/clases clave.        |

Puntos extra (opcionales):

- **Tests** con unittest o pytest (al menos 3 tests que pasen).
- **Visualización** con matplotlib o pandas.plot.
- **Interfaz** más allá de consola (tkinter, streamlit, flask).
- **Entorno reproducible** con uv o requirements.txt.
- **Herencia** o composición entre múltiples clases.

### 4. Estructura Sugerida del Repositorio

```

proyectofinal-nombre-del-grupo /
  main.py           – punto de entrada; se ejecuta con: python main.py
  README.md         – descripción, instalación, uso, ejemplos
  requirements.txt  – dependencias (o pyproject.toml si usás uv)
  src/              – módulos propios (.py) organizados por responsabilidad
  data/             – archivos de entrada/salida (.csv, .json, .txt)
  tests/            – tests opcionales
  .gitignore        – ignora __pycache__, .venv, .env, etc.
```

El programa **debe correr** con `python main.py` en una computadora limpia siguiendo las instrucciones del README. Si el profesor no puede ejecutarlo, se considera no entregado.

## 5. Entrega

---

La entrega tiene **dos partes obligatorias**:

### 5.1. Parte 1: Código en GitHub (antes del 14 de mayo, 09:00 h)

1. Repositorio **público** en GitHub con el código final. **Un solo repo por grupo**; todos los integrantes como colaboradores.
2. README.md con: **nombres completos de los integrantes**, descripción del problema, instalación, uso, capturas o ejemplo de salida.
3. Historial de *commits* reales distribuidos a lo largo de al menos **una semana** —no todo en un solo *push*. Debe haber commits de **todos** los integrantes.
4. **Uno** de los integrantes sube el **link del repo** a MIU (basta con uno por grupo).

### 5.2. Parte 2: Presentación en vivo (14 de mayo, Auditorio Hayek)

- **7–10 minutos** de presentación + **5 minutos** de preguntas en vivo.
- **Todos los integrantes hablan** durante la presentación (se reparten las partes).
- Traigan una laptop cargada; van a proyectar y **demostrar el programa corriendo**.
- Estructura sugerida:
  - ¿Qué problema resolvés? (1 min)
  - *Demo* en vivo del programa funcionando (2–3 min)
  - Recorrido por el código: decisiones clave y conceptos del curso aplicados (2–3 min)
- Podemos hacerte preguntas sobre **cualquier línea** del código. Si no podés explicarla, no cuenta.

**Llegadas tarde:** el orden se asigna antes del evento. Si no estás presente cuando te toca, pasás al final de la cola; si no llegás, **no hay presentación** y se pierden los puntos asociados.

## 6. Rúbrica de Evaluación

---

**Punteo total: 30 puntos**

| Criterio                           | Qué se evalúa                                                                                               | Pts        |
|------------------------------------|-------------------------------------------------------------------------------------------------------------|------------|
| <b>Problema &amp; alcance</b>      | El problema está claro y el programa lo resuelve. No es trivial ni exagerado.                               | /3         |
| <b>POO (clases)</b>                | Uso correcto de al menos 1 clase propia, con atributos y métodos que tienen sentido.                        | /4         |
| <b>Estructura y modularidad</b>    | Código dividido en funciones/módulos con nombres descriptivos. Nada de un solo archivo monolítico ilegible. | /3         |
| <b>Archivos y/o API</b>            | Lectura/escritura de archivos o consumo de API funciona sin reventar.                                       | /3         |
| <b>Manejo de errores</b>           | <code>try/except</code> donde importa. El programa no se rompe con entradas inválidas o archivos faltantes. | /2         |
| <b>Git &amp; documentación</b>     | Commits reales distribuidos. <b>README</b> claro. Docstrings donde aportan.                                 | /3         |
| <b>Funcionamiento end-to-end</b>   | El programa corre sin intervención, produce resultado útil, no tiene pantallas rotas.                       | /4         |
| <b>Presentación en vivo</b>        | Demo fluido, explicación clara del problema y la solución en el tiempo asignado.                            | /4         |
| <b>Defensa (preguntas en vivo)</b> | Podés explicar decisiones de diseño y cualquier línea del código.                                           | /4         |
| <b>TOTAL</b>                       |                                                                                                             | <b>/30</b> |

**Puntos extra (hasta +3):** tests que pasan, visualizaciones, interfaz gráfica/web, herencia bien usada, entorno reproducible con uv. *No sustituyen* los requisitos mínimos.

## 7. Cronograma Sugerido

Trabajá en el proyecto **todos los días** entre el 23 de abril y el 14 de mayo. Un commit diario es mejor que cinco commits la noche anterior.

| Semana           | Qué hacer                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>23–30 abr</b> | Elegir problema, crear repo GitHub, diseñar clases y funciones principales. Código base con menú/flujo principal.         |
| <b>1–7 may</b>   | Implementar lectura/escritura de archivos o consumo de API. Agregar manejo de errores. Probar casos límite.               |
| <b>8–13 may</b>  | Pulir UX, mejorar <b>README</b> , documentar funciones/clases, preparar demo y presentación. Ensayar <b>en voz alta</b> . |
| <b>14 may</b>    | Presentación y defensa en Auditorio Hayek.                                                                                |

## 8. Política de Integridad Académica

- El trabajo es **en grupos de 2 o 3 personas**. Todos los integrantes **presentan** y responden preguntas en la defensa.
- Cada integrante debe tener **commits propios en GitHub**. Un solo miembro empujando todo el código = todos pierden los puntos de Git.

- Usar AI (ChatGPT, Claude, Copilot, Cursor) es **permitido y esperado**, pero **debes entender y poder explicar cada línea**.
- Durante la defensa, podemos pedirte que **modifiques algo en vivo** (ej. “agregá un filtro por fecha”, “cambiá el formato de salida a JSON”). Si no podés guiar el cambio, la nota baja.
- Copiar el proyecto de otra persona (o forkear un repo existente sin aportar cambios sustanciales) = **cero** para todos los involucrados.
- Proyectos idénticos entre estudiantes, aunque sean de secciones distintas, se tratan como plagio.